

Operating System

Lecture-1 • 14.07.2026

Operating System (OS) is a system software that operates the system.

- Interface between user and hardware
- Memory Management
- CPU Scheduling

Examples: macOS, Linux, Windows, Android, iOS.

Lecture-2 • 19.07.2026

What is an Operating System?

An Operating System (OS) is a program that acts as an intermediary between a computer user and the computer hardware.

OS is software that controls the operation of computers and directs the processing of programs.

Goals:

- Execute user programs.
- Make the computer system convenient and user-friendly to use.
- Use computer hardware in an efficient manner.

Computer System Structure

There are 4 main components:

- Hardware: Provides basic computing resources.

Example: CPU, Memory, I/O devices.

- Operating System: Controls and coordinates use of hardware among various applications and users.

Example: macOS, Windows, Linux.

- Application Programs: Define how system resources are used to solve users' computing problems.

Example: Compilers, Web browsers, video games, etc.

- Users: Entities interacting with the system.

Example: People, machines, other computers.

System Perspective

What an OS does depends on the user's point of view. There are 2 points of view:

- User View: Ease of use and good performance.
- System View: Efficient resource allocation and control program.

Types of Systems by Usage:

- Single-User System: Prioritize convenience, ease of use, and good performance; resource utilization is not a primary concern.
- Shared Computers: Focus on maximizing resource sharing so all concurrent users get fair access.
- Workstations: Balance individual usability with sharing centralized resources from network servers.
- Handheld/Mobile Devices: Focus on battery conservation and user responsiveness under limited hardware resources.
- Embedded Systems: Found inside cars, appliances, or machinery. Focused entirely on background operations.

Formal Definitions of OS Functions

- Resource Allocator: Manages system resources and resolves competing demands fairly and efficiently.
- Control Program: Controls execution of programs to prevent errors and improper use of the computer.
- Kernel: The primary software component that remains loaded and actively running in memory at all times.
- Middleware: A set of software frameworks that provide additional services to application developers.

Bootstrap / Bootloader

- Startup Role: A small initialization program loaded automatically when a computer powers up or reboots.
- Storage Location: Stored permanently in non-volatile memory (ROM or EPROM), commonly known as firmware chips.
- Tasks Executed:
 - Initializes hardware components across the board.
 - Locates and loads the OS kernel into main memory to launch the OS.

Steps: Startup Operation (Bengali Notes from Source):

- CPU on ROM ■■■■■■ Instruction ■■■■■■
 - ⇒ ROM ■■■■■■ software ■■■■■■ ■■■■■■ firmware
 - ⇒ Firmware ■■■■■■ ■■■■■■ BIOS (Basic Input Output System)
 - ⇒ BIOS ■■■ code ■■■■■■ ■■■■■■ initialize ■■■ active ■■■■■■
 - ⇒ OS secondary memory ■■■ ■■■■■■ ■■■■■■ H.D. ■■■■■■ OS ■■■ copy RAM-■■■■■■■

Computer System Organization

- Hardware Interconnection: One or more CPUs together with various device controllers connect through a single common bus providing access to shared memory.
- Concurrent Execution: Hardware devices and processors operate at the same time, competing for available memory access cycles.

Lecture-3 • 26.07.2026

Types of Operating Systems

Batch Operating System

A system designed for non-interactive tasks where jobs are executed automatically without direct user interaction.

- Batching Concept: To speed up overall execution, individual user jobs requiring similar software or setup are grouped together into batches.
- How it works: The system holds queued jobs in memory and processes them sequentially in First-Come, First-Served (FCFS) order. When a job is completed, its allocated memory is freed and its output is redirected to an output spool for later printing.

Workflow:

[User Jobs] → Submit to [Operator] (Jobs collected into batches) → [Batches processed through Computer]

Examples:

- Telecom Billing: Batch processing monthly telephone call records to generate customer bills.
- Bank Processing: Running overnight processing for international transfers or clearing accounts.
- Corporate Payroll Systems: Calculating monthly salaries and printing employee pay slips.

Advantages:

- **Faster Group Execution:** Similar jobs in the batch are executed at higher speed.
- **No User Overhead:** Automated transitions mean the next job begins running instantly once the active job finishes.
- **Better CPU Utilization:** Minimizes CPU idle time.
- **Task Partitioning:** Large batch processes are divided into smaller sub-processes for better handling.

Disadvantages:

- Difficult to debug.
- Infinite loop risk.
- Batch systems are costly.

Time-Sharing Operating System

Definition: A system model that allows multiple users at different terminals to access a single central computer at the same time.

Time Quantization: The CPU assigns an equal fixed portion of time (called a time quantum/slice) to each task so every process runs smoothly.

Time Quantum (TQ) = $n \text{ min}$

U1 (10 min) → 7 min → 4 min → 1 min → done

U2 (15 min) → 12 min → 9 min → 6 min → 3 min → done

Um (3 min) → 0 min → done

Characteristics:

- Multi-user system
- Fixed time slices
- Fair CPU sharing
- **Multitasking:** Each user receives a portion of CPU time, making these systems effectively known as multitasking systems. Time-sharing is considered a logical step forward from basic multiprogramming.

Examples: MULTICS, UNIX, etc.

Distributed Operating System

Definition: A system whose individual components are spread across different networked computers that communicate by passing messages.

Examples: LOCUS, AEGIS, etc.

Interconnected autonomous computers run together where each individual system has its own CPU and memory unit. This is called a loosely coupled system.

Coupled Systems:

- **Tightly Coupled Systems:** Processors share a single main memory and communicate directly over a high-speed bus.

Examples: Parallel Processing, Shared Memory.

- **Loosely Coupled Systems:** Independent computers have separate local memory units and CPUs, exchanging messages over a communication network.

Network Operating System (NOS)

Definition: An operating system running on a network host that enables shared access to files, printers, security services, and applications across a private network.

Examples: Windows Server 2003/2008, UNIX, Linux, Mac OS X, Novell NetWare, BSD.

Distributed OS vs Network OS

<th>Feature
<th>Distributed OS
<th>Network OS
<td>Definition
<td>Multiple computers work together as one system.
<td>Multiple independent computers connected by a network.
<td>User View
<td>Appears as a single computer.
<td>Users know they are using different computers.
<td>System Image
<td>Single System Image (SSI).
<td>No single system image.
<td>Transparency
<td>High; location of resources is hidden.
<td>Low; users explicitly access remote resources.
<td>Operating System
<td>Cooperating distributed OS across nodes.
<td>Each computer runs its own OS independently.
<td>Scalability
<td>Excellent.
<td>Good, but manual administration needed.
<td>Administration
<td>Appears centralized.
<td>Each machine is monitored separately.
<td>Examples
<td>Amoeba, Plan 9, Sprite.
<td>Windows Server, Ubuntu Server, macOS X.

Mobile & Embedded Systems

- Mobile Operating System:
 - Target Hardware: Designed specifically for handheld smartphones and tablets.
 - Special Features: Includes built-in support for hardware sensors like GPS and gyroscopes, allowing advanced apps such as augmented reality.
 - Connectivity: Connects using cellular data networks or IEEE 802.11 wireless (Wi-Fi).
 - Market Leaders: Apple iOS, Google Android.
- Embedded Operating System:
 - Definition: Specialized operating systems designed for computers built into larger machinery or consumer appliances.

- Applications: Vehicles, traffic signals, smart TVs, ATMs, flight controls, Point of Sale (POS), GPS receivers, digital cameras.

Real-Time Operating System (RTOS)

Definition: An operating system designed to guarantee that specific tasks complete within strict, predefined time limits.

Core Requirement: Operations must produce correct logic within strict deadline constraints.

Applications: Industrial machinery, medical imaging, flight control, weapon control systems, scientific experiments.

Lecture-4 • 28.07.2026

Storage Device Hierarchy

Registers $\uparrow\downarrow$ Cache $\uparrow\downarrow$ Main Memory $\uparrow\downarrow$ Solid-State Disk $\uparrow\downarrow$ Hard Disk $\uparrow\downarrow$ Optical Disk $\uparrow\downarrow$ Magnetic Tapes

(Top: Fastest, smaller, volatile • Bottom: Slower, larger, non-volatile)

Caching

Concept: Data currently in use is temporarily copied from slower main storage to faster memory to speed up overall execution.

Migration of Data 'A' from Disk to Register:

Hard Disk (A) $\rightarrow$ Main Memory (A) $\rightarrow$ Cache (A) $\rightarrow$ Hardware Register (A)

- Multitasking Environments: Must be careful to use the most recent value, no matter where it is stored in the storage hierarchy.
- Multiprocessor Environments: Must provide cache coherency in hardware such that all CPUs have the most recent value in their cache.
- Cache Coherency: Prevents cache inconsistency. The MESI algorithm is an example used in cache coherency.

Computer-System Architecture

Most systems use a single general-purpose processor.

Single vs Multiprocessor: Most legacy computers use one main CPU, but multiprocessor systems (also known as parallel systems or tightly coupled systems) are widely used today.

Key Advantages:

- Increased Throughput: Accomplishes more work in less time.
- Economy of Scale: Sharing hardware resources costs less than separate standalone computers.
- Increased Reliability: If one processor fails, the system continues running at a reduced pace (graceful degradation).

Two Types:

- Asymmetric Multiprocessing: Each processor is assigned a specific task.
- Symmetric Multiprocessing (SMP): Every processor performs all tasks without a master-slave restriction.

Dual-Mode Operation

Purpose: Allows the operating system to protect itself and other running programs from faulty or malicious user applications.

Hardware Modes:

- User Mode (Mode bit = 1): Executes standard user application code with restricted privileges.

- Kernel Mode (Mode bit = 0): Executes core OS code with full access to hardware; privileged instructions run only here.

Mode Switch Workflow:

- User process is executing in User Mode (mode bit = 1).
- User process makes a System Call → triggers a Trap.
- Mode bit is set to 0 (switches to Kernel Mode).
- Kernel executes the requested system call.
- System call completes → mode bit is reset to 1 → returns control back to User Mode.

Chapter-2 • Operating-System Structures • Lecture-5 • 02.08.2026

An operating system provides an environment for executing programs and offering services to users, processes, and other systems. These services are split into two groups: those that help users do their work, and those that keep the computer running safely and fast.

Operating System Services

1. User-Focused Services

- User Interface (UI): Almost all OSs feature a UI to allow interaction.
- Command-Line Interface (CLI)
- Graphical User Interface (GUI)
- Batch Interface
- Program Execution: Loads a program into memory, runs it, and ends its execution normally or with an error.
- I/O Operations: Handles input/output requests required by running programs involving I/O devices.
- File-System Manipulation: Allows programs to read, write, create, delete, and search files or directories; handles file permissions.
- Communication: Enables processes to exchange information on the same computer or across network-connected computers via:
 - Shared Memory
 - Message Passing
- Error Detection: Continuously monitors the system for potential errors to ensure correct and consistent computing (CPU, main memory, I/O devices, or user programs). Includes debugging facilities.

2. System-Focused Services

Operate in the background to ensure resource sharing, efficiency, and system security:

- Resource Allocation: Allocates resources among multiple users or simultaneous parallel tasks. Uses special allocation code for CPU cycles, main memory, and file storage; release code for I/O devices.
- Accounting: Tracks which users use how many and what kinds of computer resources.
- Protection and Security:
 - Protection: Controls access to system resources and prevents concurrent processes from interfering with each other.
 - Security: Authenticates users to defend systems and external I/O devices against unauthorized access from outsiders.

System Calls & APIs

System Calls

- Definition: A programming interface that provides user-level processes a way to request OS services.
- Mechanism: An explicit request to the OS Kernel made via a software interrupt from user processes.
- Implementation: Typically written in high-level languages like C or C++.

Application Programming Interface (API)

- Definition: A set of functions, objects, protocols, or data structures that support application development for programmers.
- Role: APIs sit as a library or layer between regular programs and the rest of the OS.
- Function: APIs handle the low-level mechanism of passing parameters/information to the kernel and switching execution to supervisor (kernel) mode.
- Common APIs:
 - Win32 API (for Windows)
 - POSIX API (for POSIX-based systems, including Linux, UNIX, macOS)
 - Java API (for Java Virtual Machine – JVM)

POSIX API

POSIX (Portable Operating System Interface) is a standardized set of APIs designed to maintain software compatibility between different operating systems. By adhering to POSIX standards, developers can write application code in C/C++ that runs across multiple Unix-like systems without system-specific rewrites. It acts as a bridge wrapping low-level system calls into standardized function calls.

How System Calls are Implemented

- System Call Numbering: Each system call has a unique number assigned to it.
- System Call Table: An indexed table maintained by the system call interface mapping numbers directly to corresponding kernel functions.
- Execution Flow:
 - User application calls an API function in user mode.
 - System call interface intercepts it, looks up the system call number, and switches to kernel mode.
 - Kernel executes the actual system call implementation.
 - System call interface returns operation status and return values back to user mode.

Example: Sequence to copy contents from Source File to Destination File:

- 1. Acquire Input File Name (display prompt, read input)**
- 2. Acquire Output File Name (display prompt, read input)**
- 3. Open Input File (abort if file does not exist)**
- 4. Create Output File (abort if file already exists)**
- 5. Copy Loop: Read data from input file and write to output file until read fails**
- 6. Clean Up & Exit: Close both files, print completion message, and terminate**

Operating System Design & Implementation

- Design Goals:
 - User Goals: Convenient, easy to use, easy to learn, reliable, safe, and fast.
 - System Goals: Easy to design, implement, and maintain; flexible, reliable, error-free, and efficient.
- Mechanisms vs. Policies:

- Mechanisms: Determine how to do something (e.g., timer routine for context switching).
- Policies: Determine what will be done (e.g., deciding which process gets the CPU next and for how long).
- Implementation Languages:
- Early OS: Written entirely in Assembly Language (fast, direct hardware access, but difficult to debug and high line count).
- Modern OS: Written mostly in algorithmic languages (C/C++) with small amounts of assembly for critical routines.

Pros: Code written faster, easier to read/debug/maintain, compact, portable.

Cons: Slight reduction in execution speed and higher memory usage.

Lecture-6 • 04.08.2026

Operating System Structure

Refers to how the OS is organized internally to meet goals like high performance, modularity, security, maintainability, and portability.

1. Simple Structure

- Early, small operating systems designed with minimal structure due to hardware limitations.
- Example: MS-DOS.
- Characteristics: Not divided into distinct modules; lacks dual-mode operation and hardware protection; difficult to extend; highly vulnerable to malicious user programs; if a user program fails, the entire system crashes.
- Structure: Application Program → Resident System Programs → MS-DOS Device Drivers → ROM BIOS Device Drivers.

2. Monolithic Structure

- The entire OS (file system, memory management, process scheduling, drivers) runs as a single massive program in Kernel Mode.
- Example: Early UNIX, traditional Linux kernel.
- Pros: High performance, fast communication among kernel modules, efficient system call execution.
- Cons: Large kernel size, difficult debugging; a faulty driver can crash the entire OS.

3. Layered Structure

- Broken down into smaller, modular layers (Level 0 to Level N). Each layer uses functions and services of only lower-level layers.
- Layer 0: Hardware
- Layer N: User Interface
- Pros: Simplicity of construction and debugging; modular verification.
- Cons: Difficult to appropriately define each layer; less efficient due to processing overhead across layers.

4. Microkernel Structure

- Places only bare minimum system components in the kernel and moves non-essential components to user space.
- File systems, network stacks, and device drivers run as user-mode processes.
- Modules communicate via message passing through the microkernel.
- Examples: Mach, MINIX 3, QNX.
- Pros: Easier to extend, easier to port to new hardware, more reliable, highly secure.
- Cons: Performance overhead from frequent user-space to kernel-space communication.

5. Modular Architecture (Loadable Kernel Modules - LKM)

- Best current methodology: kernel has core components and links additional services dynamically via modules at boot time or runtime.
- Avoids recompiling the kernel when new features or drivers are added.
- Examples: Solaris, Linux, macOS.
- Pros: More flexible than layered approach (modules can call each other directly); more efficient than microkernel (no message passing needed).
- Cons: Memory fragmentation penalty; can impact TLB usage.

6. Hybrid Structure

Combines multiple structural approaches to address performance, security, and usability simultaneously. Most modern OSs are hybrid:

- Linux / Solaris: Monolithic core inside kernel address space for speed, paired with modular architectures for dynamic loading.
- Windows: Mostly monolithic for performance, plus microkernel-style concepts for user-mode subsystems.
- macOS / iOS: Hybrid architecture.

7. Virtual Machine

- Treats hardware and the OS kernel as if they were entirely hardware.
- Creates the illusion that each process has its own processor and virtual memory by providing a copy of the underlying computer.

8. Exokernel Structure

- Provides minimal abstraction over hardware, granting applications direct access to physical hardware resources.
- Applications manage raw resources through Library Operating Systems (LibOS).
- Stack: Applications → LibOS → Exokernel → Hardware.
- Example: MIT Exokernel, Aegis.
- Pros: Maximizes performance, highly customizable, extremely efficient resource utilization.
- Cons: Difficult to program, rarely used in commercial systems.

Chapter-3 • Process • Lecture-7 • 11.08.2026

Process Concept

Definition: A process is an active program currently in execution, unlike a passive program stored on a disk. The same program can have multiple active processes running at once.

Memory Layout / Process Sections:

- Text Section: Contains executable instructions or code.
- Data Section: Stores global variables (split into initialized and uninitialized data).
- Heap: Memory dynamically allocated at runtime (e.g., using malloc()).
- Stack: Stores temporary execution data such as function parameters, return addresses, and local variables.
- High Memory: Holds command-line arguments (argc, argv) passed to the main() function.

Process States & Transitions

- New: The process is being created.
- Ready: The process is waiting to be assigned to a CPU core.

- Running: Instructions are actively executing on the CPU.
- Waiting: The process is blocked, waiting for an I/O event or signal.
- Terminated: The process has completed its execution.

Transitions:

- New → Ready (Admitted)
- Ready → Running (Scheduler dispatch)
- Running → Ready (Interrupt)
- Waiting → Ready (I/O or event completion)
- Running → Waiting (I/O or event wait)
- Running → Terminated (Exit)

Process Control Block (PCB)

The PCB (also called Task Control Block) maintains all information needed to represent, run, or suspend a process:

- Process State: Current state (New, Ready, Running, Waiting, etc.).
- Program Counter (PC): Address of the next instruction to execute.
- CPU Registers: State registers (stack pointers, index registers) saved upon an interrupt to resume safely later.
- CPU Scheduling Information: Priorities, queue pointers, scheduling parameters.
- Memory Management Information: Base/limit registers, page tables, or segment tables.
- Accounting Information: CPU execution time, time limits, process ID numbers, accounts.
- I/O Status Information: Allocated I/O devices, open file descriptors, and buffers.

Lecture-8 • 18.08.2026

Process Scheduling

The method an OS uses to decide which process gets to run when multiple processes are ready. The process scheduler selects an eligible process from the ready queue and allocates CPU time to it.

Scheduling Queues:

- Job Queue: Holds all processes in the system; stored on secondary storage (disk) and managed by the long-term scheduler.
- Ready Queue: Holds processes residing in main memory that are ready and waiting to be allocated CPU time. Managed by the short-term scheduler.
- Wait / Device Queue: Holds processes waiting for specific events (disk I/O, child process termination, interrupt completion).

Process Execution Paths from CPU:

- I/O Request: The process issues an I/O request and is placed in an I/O wait queue; upon I/O completion, it returns to the ready queue.
- Child Process Creation: The process forks a child and waits in a wait queue until the child terminates, then returns to the ready queue.
- Interrupt / Time Slice Expiry: The process is forcibly removed from the CPU core due to an interrupt/timer expiration and placed back into the ready queue.
- Termination: Completes execution, is removed from all queues, and its PCB and resources are deallocated.

Types of Schedulers:

- **Short-Term Scheduler (CPU Scheduler):** Selects which process from the ready queue in memory should execute next on the CPU. Invoked frequently (milliseconds) and must be fast.
- **Long-Term Scheduler (Job Scheduler):** Selects which processes from the secondary storage job queue should be brought into main memory. Invoked infrequently (seconds or minutes) and controls the degree of multiprogramming.

Context Switching

The operation by which an OS saves the execution state (context) of a currently running process/thread and restores the saved state of another process/thread so the CPU can switch execution between them.

Lecture-9 • 23.08.2026

Operations on Processes

1. Process Creation

- **Process Tree:** Creating process is the parent; new processes are children. Forms a tree hierarchy (e.g., rooted at init or systemd with PID = 1 in Linux).
- **Process Identifier (PID):** A unique integer used by the OS to identify each process.
- **Resource Sharing Options:**
 - Parent and children share all resources.
 - Children share a subset of parent resources.
 - Parent and children share no resources.
- **Execution Options:**
 - Parent continues executing concurrently with children.
 - Parent waits until some or all children terminate.
- **Address Space Options:**
 - **Duplicate:** Child process is an exact copy of parent.
 - **New Program:** Child process has a completely new program loaded into its address space.
- **UNIX/Linux Implementation:**
 - **fork():** Duplicates the parent process into a child. Returns 0 to the child, and returns the child's PID (>0) to the parent.
 - **exec() / execlp():** Overwrites child address space with a new program binary.
 - **wait():** Suspends parent execution until the child terminates.

2. Process Termination

- **Normal Termination:** Process executes final statement, calls `exit()`, returns exit status data to parent via `wait()`, and OS deallocates its resources.
- **Abnormal / Parent Termination:** Parent terminates child using `abort()` because:
 - Child exceeded allocated resources.
 - Task assigned to child is no longer needed.
 - Parent is exiting and OS does not allow orphan processes.
- **Special Cases:**
 - **Cascading Termination:** OS automatically terminates all child, grandchild, and descendant processes if parent terminates.
 - **Zombie Process:** A process that has finished execution, but its parent has not yet called `wait()` to collect its exit status and free its PCB entry.

- Orphan Process: A process whose parent terminated without calling wait(). Adopted by init (or systemd), which periodically calls wait().

Interprocess Communication (IPC)

A set of mechanisms provided by the OS allowing processes to exchange data and synchronize actions.

- Independent Process: Does not share data with any other process; cannot affect or be affected by other processes.
- Cooperating Process: Can affect or be affected by other processes executing in the system.

Reasons for Cooperating Processes:

- Information Sharing: Multiple processes access shared data.
- Computation Speedup: Breaking tasks into subtasks running concurrently across multiple cores.
- Modularity: Dividing system functions into separate processes or threads.

Models of IPC:

- Shared-Memory Model: A shared memory region is established. Processes read and write directly to this region.

Advantage: Fast communication at memory-transfer speeds.

- Message-Passing Model: Communication occurs by exchanging formatted messages (using operations like send(message) and receive(message)).

Characteristics: Slower due to system call overhead, but avoids memory access conflicts.

Chapter-5 • CPU Scheduling

CPU-I/O Burst Cycle

- CPU Burst: Period when the process is actively executing instructions on the CPU (arithmetic, loads, stores).
- I/O Burst: Period when the process is idle with respect to the processor, waiting for an I/O transfer to complete (disk, network).

Process execution alternates between CPU and I/O bursts, terminating on a final CPU burst with a system request to exit.

Basic Concepts

- Ready Queue: Keeps track of processes residing in main memory ready to execute. Can be FIFO, priority queue, balanced tree, or unordered list.
- Preemptive Scheduling: OS can interrupt and forcibly remove the CPU from an executing process.
- Non-Preemptive Scheduling: Process holds CPU until it voluntarily releases it (by terminating or waiting for I/O).

Scheduling Decisions occur when a process:

- Switches from running to waiting state (e.g., I/O request).
- Switches from running to ready state (e.g., interrupt occurs).
- Switches from waiting to ready state (e.g., I/O completes).
- Terminates.

Lecture-10 • 30.08.2026

Dispatcher

The dispatcher is the module that gives control of the CPU core to the process selected by the CPU scheduler.

Actions performed:

- Context switching from one process to another.
- Switching to User Mode.
- Jumping to the proper location in the user program to resume execution.

Dispatch Latency: The time required for the dispatcher to stop one process and start another running. Must be as fast as possible.

CPU Scheduling Criteria

- CPU Utilization: Percentage of time CPU is active (ranges conceptually from 0% to 100%; real systems typically 40% to 90%).
- Throughput: Number of processes completed per unit time.
- Turnaround Time: Total time elapsed from submission to completion (waiting + executing + I/O).
- Waiting Time: Cumulative time spent waiting in the ready queue.
- Response Time: Time from submission until the first response is produced.

5.3 Scheduling Algorithms & Solved Problems

1. First-Come, First-Served (FCFS)

Case-1: Arrival Time = 0 for all

<th>Process	<th>Burst Time (BT)
<td>P1	<td>24
<td>P2	<td>3
<td>P3	<td>3

Gantt Chart:

P1

P2

P3

0 24 27 30

Average Waiting Time (AWT) = $(0 + 24 + 27) / 3 = 51 / 3 = 17$

Case-2: Different Arrival Times

<th>Process	<th>Arrival Time (AT)	<th>Burst Time (BT)
<td>P1	<td>2	<td>2
<td>P2	<td>0	<td>1
<td>P3	<td>2	<td>3
<td>P4	<td>3	<td>5
<td>P5	<td>4	<td>2

Gantt Chart:

P2

Idle

P1

P3

P5

P4

0 1 2 4 6 8 11

$$AWT = [(2 - 2) + (0 - 0) + (4 - 2) + (6 - 4) + (8 - 3)] / 5 = (0 + 0 + 2 + 2 + 5) / 5 = 1.2$$

Lecture-11 • 01.09.2026

2. Shortest Job First (SJF)

Why SJF is Better than FCFS:

- Provably Minimal Average Waiting Time: Gives the smallest possible average waiting time compared to any other scheduling algorithm.
- Prioritizing Shorter Jobs: Serving quick tasks first helps everyone finish sooner on average.
- Elimination of Convoy Effect: FCFS creates a convoy effect where small tasks wait behind one long task. SJF runs quick tasks immediately.

Case-1: Non-Preemptive SJF (Arrival Time = 0)

Processes: P1 (BT = 24), P2 (BT = 3), P3 (BT = 3)

Gantt Chart:

P2

P3

P1

0 3 6 30

$$AWT = (0 + 3 + 6) / 3 = 9 / 3 = 3$$

Case-2: Non-Preemptive SJF with Arrival Times

<th>Process	<th>AT	<th>BT
<td>P1	<td>2	<td>2
<td>P2	<td>0	<td>1
<td>P3	<td>5	<td>3
<td>P4	<td>3	<td>5
<td>P5	<td>4	<td>2

Gantt Chart:

P2

Idle

P1

P5

P3

P4

0 1 2 4 6 9 14

Note from source: ■■■■■■ ■■■■■■■■■■ ■■■■■■ ■■■■ (Both FCFS/SJF yield same initial order here)

$$AWT = [(0 - 0) + (2 - 2) + (4 - 4) + (6 - 5) + (9 - 3)] / 5 = (0 + 0 + 0 + 1 + 6) / 5 = 7 / 5 = 1.4$$

3. Shortest Remaining Job First (SRJF / Preemptive SJF)

Why SRJF is better than Non-preemptive SJF: Dynamic adaptation to late arrivals of shorter jobs.

<th>Process	<th>AT	<th>BT
<td>P1	<td>0	<td>10

<td>P2	<td>2	<td>2
<td>P3	<td>3	<td>1
<td>P4	<td>1	<td>3
<td>P5	<td>4	<td>2

Gantt Chart:

P1

P4

P2

P3

P5

P4

P1

0 1 2 4 5 7 9 18

Waiting Times:

$$WT1 = 18 - 0 - 10 = 8$$

$$WT2 = 4 - 2 - 2 = 0$$

$$WT3 = 5 - 3 - 1 = 1$$

$$WT4 = 7 - 1 - 3 = 3$$

$$WT5 = 9 - 4 - 2 = 3$$

$$AWT = (8 + 0 + 1 + 3 + 3) / 5 = 17 / 5 = 3.4$$

Lecture-12 • 06.09.2026

4. Priority Scheduling

Divided into Preemptive and Non-preemptive.

Rules from Source:

- priority value Low integer = high priority (e.g., Priority 1 is higher than Priority 5).
- Non-preemptive- interrupt .
- Preemptive- Emergency (high priority) execute .

A. Non-Preemptive Priority Scheduling

<th>Process	<th>AT	<th>BT	<th>Priority
<td>P1	<td>3	<td>5	<td>2
<td>P2	<td>0	<td>2	<td>3
<td>P3	<td>2	<td>4	<td>1
<td>P4	<td>5	<td>2	<td>6
<td>P5	<td>10	<td>3	<td>5

Gantt Chart:

P2

P3

P1

P5

P4

0 2 6 11 14 16

$$AWT = [(6 - 3) + (0 - 0) + (2 - 2) + (14 - 5) + (11 - 10)] / 5 = (3 + 0 + 0 + 9 + 1) / 5 = 13 / 5 = 2.6$$

B. Preemptive Priority Scheduling

<th>Process	<th>AT	<th>BT	<th>Priority
<td>P1	<td>1	<td>5	<td>2
<td>P2	<td>0	<td>2	<td>3
<td>P3	<td>2	<td>4	<td>1
<td>P4	<td>5	<td>2	<td>6
<td>P5	<td>10	<td>3	<td>5

Gantt Chart:

P2

P1

P3

P1

P2

P5

P4

0 1 2 6 10 11 14 16

Waiting Times:

$$P1 = 10 - 1 - 5 = 4$$

$$P2 = 11 - 0 - 2 = 9$$

$$P3 = 6 - 2 - 4 = 0$$

$$P4 = 16 - 5 - 2 = 9$$

$$P5 = 14 - 10 - 3 = 1$$

$$AWT = (4 + 9 + 0 + 9 + 1) / 5 = 23 / 5 = 4.6$$

5. Starvation, Aging & Round Robin (RR)

Starvation & Aging

- Starvation (Indefinite Blocking): A major problem with priority scheduling where low-priority processes remain blocked indefinitely because a continuous stream of higher-priority processes keeps arriving.
- Aging: The primary solution to starvation. It gradually increases the priority of processes waiting in the system over time until even the lowest-priority process advances enough to be executed.

Round Robin Scheduling

- Similar to time-sharing systems.
- Each process is allocated a fixed Time Quantum (TQ).
- By nature, it is preemptive.
- Maintains a circular Ready Queue structure.

Example Problem: Time Quantum (TQ) = 3

<th>Process	<th>AT	<th>BT	<th>Remaining Time (RT)
-------------	--------	--------	-------------------------

<td>P1	<td>0	<td>5	<td>5 → 2 → 0
<td>P2	<td>1	<td>4	<td>4 → 1 → 0
<td>P3	<td>2	<td>2	<td>2 → 0
<td>P4	<td>3	<td>1	<td>1 → 0

Gantt Chart:

P1

P2

P3

P4

P1

P2

0 3 6 8 9 11 12

Exam Tip from Notes:

Answer ■■■■■■ ■■■■■ Gantt Chart ■■■■■■ ■■■ ■■ ■■■■■■ ■■■■■■■■■■■■ ■■■■■ ■■■■■ answer ■■■■■
■■■■■